

Data Types, Graphical Marks, and Visual Encoding Channels

A visualization represents data using a collection of *graphical marks* such as bars, lines, and point symbols. The attributes of a mark — such as its position, shape, size, or color — serve as *channels* in which we can encode underlying data values.



With a basic framework of *data types*, *marks*, and *encoding channels*, we can concisely create a wide variety of visualizations. In this notebook, we explore each of these elements and show how to use them to create custom statistical graphics.

Global Development Data

We will visualize global health and population measures for countries of the world, recorded over the years 1955 to 2005. The data was collected by the [Gapminder Foundation](#) and shared in [Hans Rosling's popular TED talk](#). (If you haven't seen the talk, we encourage you to watch it!)

Let's first load the dataset from the `vega-datasets` collection into a JavaScript array.

```
data = ► Array(693) [Object, Object, Object, Object, Object, Object, Object, Object, Obje
```

```
data = (await require('vega-datasets@1'))['gapminder.json']()
# now dig in the data:
```

693 rows, 6 columns!

```
md`${data.length} rows, ${Object.keys(data[0]).length} columns!`
```

year	country	cluster	pop	life_expect	fertility
1955	"Afghanistan"	0	8891209	30.332	7.7
1960	"Afghanistan"	0	9829450	31.997	7.7
1965	"Afghanistan"	0	10997885	34.02	7.7

year	country	cluster	pop	life_expect	fertility
1970	"Afghanistan"	0	12430623	36.088	7.7
1975	"Afghanistan"	0	14132019	38.438	7.7
1980	"Afghanistan"	0	15112149	39.854	7.8
1985	"Afghanistan"	0	13796928	40.822	7.9
1990	"Afghanistan"	0	14669339	41.674	8
1995	"Afghanistan"	0	20881480	41.763	8
2000	"Afghanistan"	0	23898198	42.129	7.4792

For each country and year (in 5-year intervals), we have measures of fertility in terms of the number of children per woman (*fertility*), life expectancy in years (*life_expect*),

```
printTable(data.slice(0, 10))
```

We also see a `cluster` field with an integer code. What might this represent? We'll try and solve this mystery as we visualize the data!

Let's also create a smaller data array, filtered down to values for the year 2000 only:

```
data2000 = ▶ Array(63) [Object, Object, Object, Object, Object, Object, Object, Object, 0
```

year	country	cluster	pop	life_expect	fertility
2000	"Afghanistan"	0	23898198	42.129	7.4792
<pre>data2000 = data.filter(d => d.year === 2000)</pre>					
2000	"Aruba"	3	69539	73.451	2.124
2000	"Australia"	4	19164620	80.37	1.756
2000	"Austria"	1	8113413	78.98	1.382
2000	"Bahamas"	3	290075	71.068	2.1111
2000	"Bangladesh"	0	130406594	62.013	3.224
2000	"Barbados"	3	273483	75.97	1.5
2000	"Belgium"	1	10263618	78.32	1.638
2000	"Bolivia"	3	8152620	63.883	3.9585

```
printTable(data2000.slice(0, 10))
```

The first ingredient in effective visualization is the input data. Data values can represent different forms of measurement. What kinds of comparisons do those measurements support? And what kinds of visual encodings then support those comparisons?

We will start by looking at the basic data types that Vega-Lite uses to inform visual encoding choices. These data types determine the kinds of comparisons we can make, and thereby guide our visualization design decisions.

Nominal (N)

Nominal data — also called *categorical* data — consist of category names.

With nominal data we can compare the equality of values: *is value A the same or different than value B?* ($A = B$), supporting statements like “A is equal to B” or “A is not equal to B”. In the dataset above, the country field is nominal.

When visualizing nominal data we should readily perceive if values are the same or different: position, color hue (blue, red, green, *etc.*), and shape are all reasonable options. In contrast, using a size channel to encode nominal data might mislead us, suggesting rank-order or magnitude differences among values that do not exist!

Ordinal (O)

Ordinal data consist of values that have a specific ordering.

With ordinal data we can compare the rank-ordering of values: *does value A come before or after value B?* ($A < B$), supporting statements like “A is less than B” or “A is greater than B”. In the dataset above, we can treat the year field as ordinal.

When visualizing ordinal data, we should perceive a sense of rank-order. Position, size, or color value (brightness) might be appropriate, whereas color hue (which is not perceptually ordered) would be less appropriate.

Quantitative (Q)

With *quantitative* data we can measure numerical differences among values. There are multiple sub-types of quantitative data:

For *interval* data we can measure the distance (interval) between points: *what is the distance to value A from value B?* ($A - B$), supporting statements such as “A is 12 units away from B”.

For *ratio* data the zero point is meaningful, so we can also measure proportions or scale factors: *value A is what proportion of value B?* (A / B), supporting statements such as “A is 10% of B” or “B is 7 times larger than A”.

In the dataset above, year is a quantitative interval field (the value of year “zero” is subjective), whereas fertility and life_expect are quantitative ratio fields (zero is meaningful for calculating proportions). Vega-Lite represents quantitative data, but does not make a distinction between interval and ratio types.

Quantitative values can be visualized using position, size, or color value, among other channels. An axis with a zero baseline is essential for proportional comparisons of ratio values, but can be safely omitted for interval comparisons.

Temporal (T)

Temporal values measure time points or intervals. This type is a special case of quantitative values (timestamps) with rich semantics and conventions (i.e., the [Gregorian calendar](#)).

Example temporal values include date strings such as “2019-01-04” and “Jan 04 2019”, as well as standardized date-times such as the [ISO date-time format](#): “2019-01-04T17:50:35.643Z”. There are no temporal values in our global development dataset above, as the year field is encoded as an integer.

The temporal type in Vega-Lite supports reasoning about time units (year, month, day, hour, etc.), and provides methods for requesting specific time intervals. For more details about temporal data in Vega-Lite, see the [TimeUnit documentation](#).

Summary

These data types are not mutually exclusive, but rather form a hierarchy: ordinal data support nominal (equality) comparisons, while quantitative data support ordinal (rank-order) comparisons.

Moreover, these data types do *not* provide a fixed categorization. For example, just because a data field is represented using a number doesn't mean we have to treat it as a quantitative type! We might interpret a set of ages (10 years old, 20 years old, *etc.*) as nominal (underage or overage), ordinal (grouped by year), or quantitative (calculate average age).

Now let's examine how to visualize these data types using *encoding channels*.

Encoding Channels

At the heart of Vega-Lite is the use of *encodings* that bind data fields (with a given data type) to available encoding *channels* of a chosen *mark* type. In this notebook we'll examine the following encoding channels:

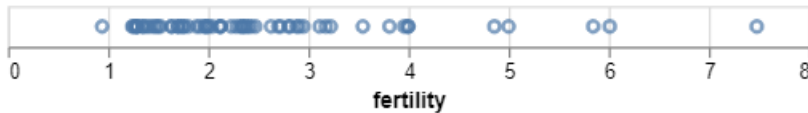
- **x**: Horizontal (x-axis) position of the mark.
- **y**: Vertical (y-axis) position of the mark.
- **size**: Size of the mark. May correspond to area or length, depending on the mark type.
- **color**: Mark color, specified as a [legal CSS color](#).
- **opacity**: Mark opacity, ranging from 0 (fully transparent) to 1 (fully opaque).
- **shape**: Plotting symbol shape for point marks.
- **tooltip**: Tooltip text to display upon mouse hover over the mark.
- **order**: Mark ordering, determines line/area point order and drawing order.
- **column**: Facet the data into horizontally-aligned subplots.
- **row**: Facet the data into vertically-aligned subplots.

For a complete list of available channels, see the [Vega-Lite encoding documentation](#).

X

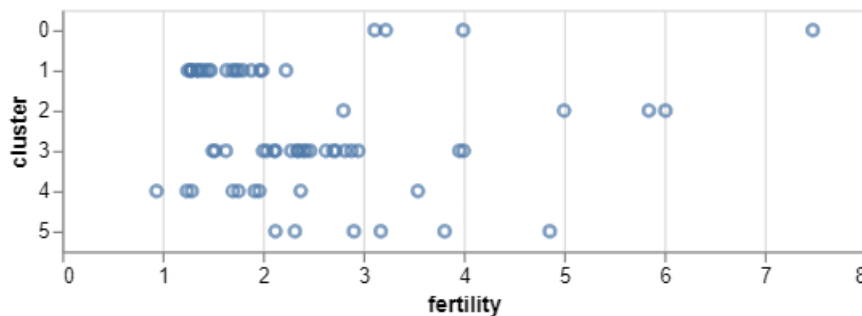
The **x** encoding channel sets a mark's horizontal position (x-coordinate). In addition,

default choices of axis and title are made automatically. In the chart below, the choice of a quantitative data type results in a continuous linear axis scale:



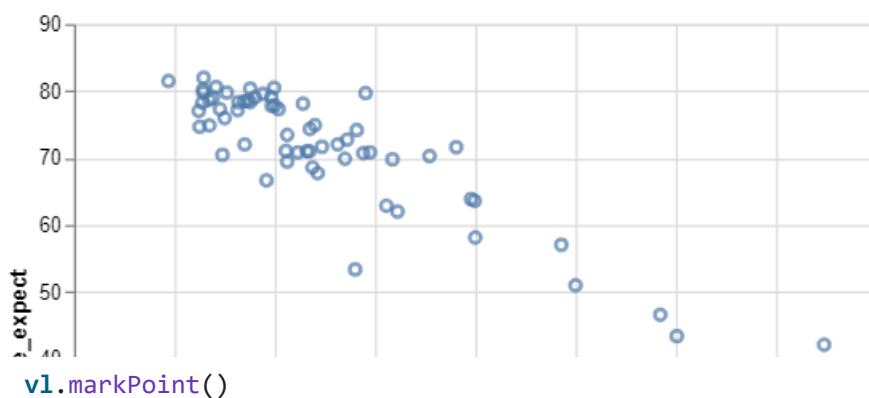
Y

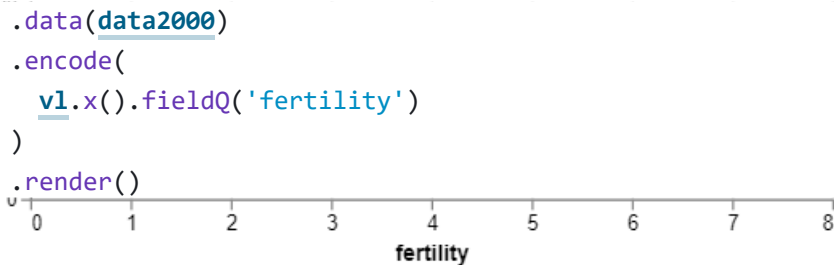
The y encoding channel sets a mark's vertical position (y-coordinate). Here we've added the `cluster` field using an ordinal (O) data type. The result is a discrete axis that includes a sized band, with a default step size, for each unique value:



What happens to the chart above if you swap the O and Q field types?

If we instead add the `life_expect` field as a quantitative (Q) variable, the result is a scatter plot with linear scales for both axes:



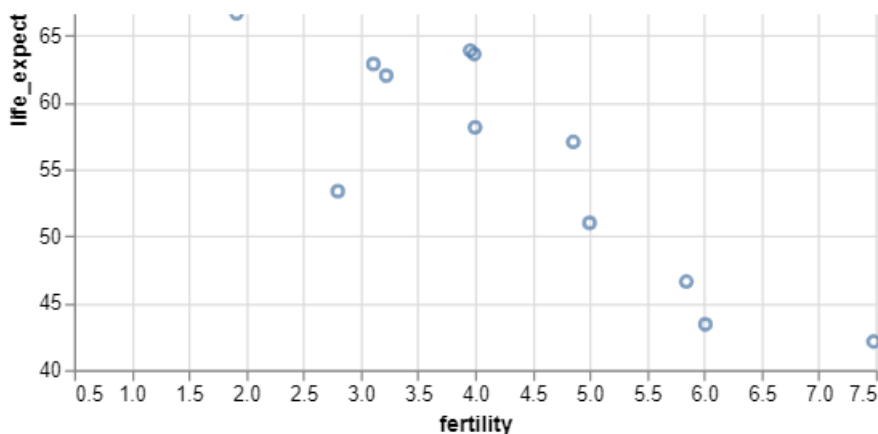


By default, axes for linear quantitative scales include zero to ensure a proper baseline for comparing ratio-valued data. In some cases, however, a zero baseline may be meaningless or you may want to focus on interval comparisons. To disable automatic inclusion of zero, configure the encoding scale attribute:

```

vl.markPoint()
.vl.data(data2000)
.vl.encode(
  vl.x().fieldQ('fertility'),
  vl.y().fieldQ('cluster')
)
.vl.render()

```



Now the axis scales no longer include zero by default. Some padding still remains, as the axis domain end points are automatically snapped to *nice* numbers like multiples of 5 or 10.

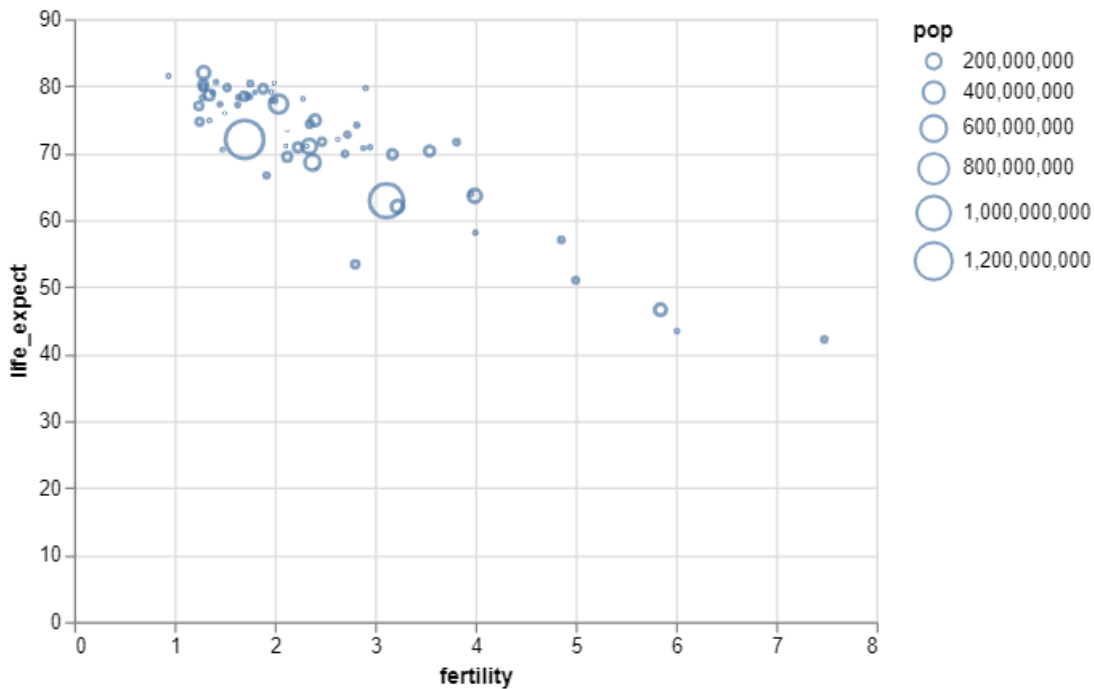
What happens if you also add `nice:false` to the scale properties above?

```

v1.markPoint()
  .data(data2000)
  .encode(
    v1.x().fieldQ('fertility'),
    v1.y().fieldQ('life_expect')
  )
  .render()

```

Let's augment our scatter plot by encoding population (pop) on the size channel. As a result, the chart now also includes a legend for interpreting the size values.

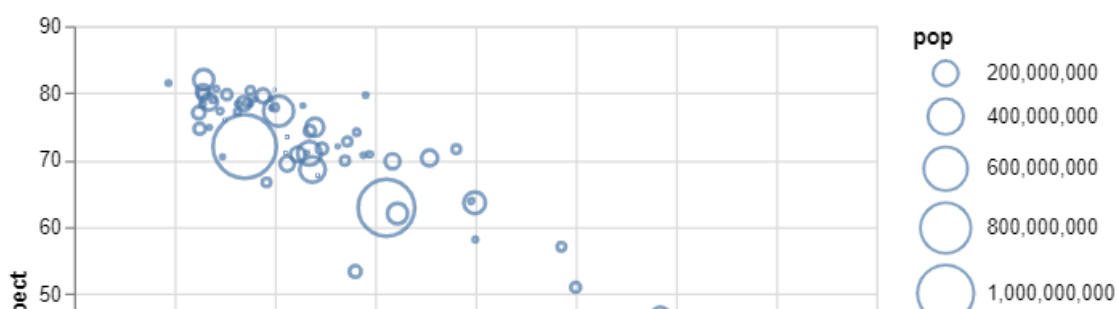


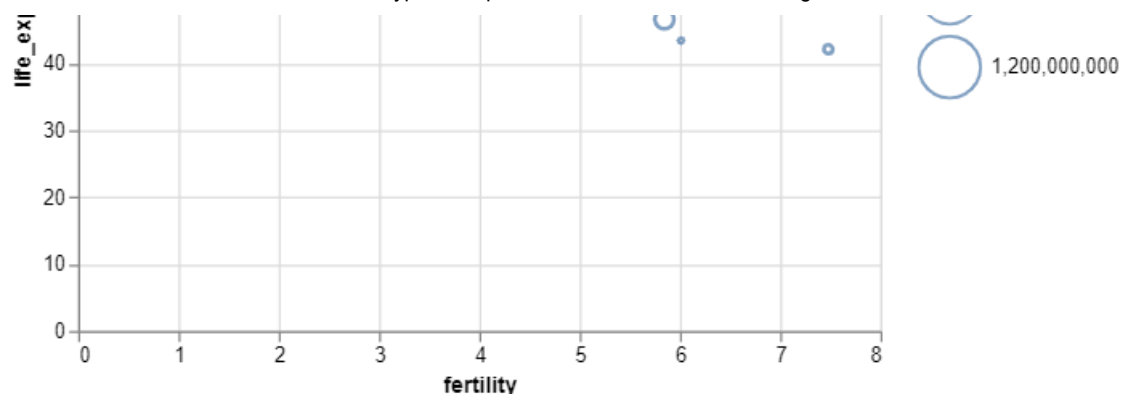
```

v1.markPoint()
  .data(data2000)
  .encode(
    v1.x().fieldQ('fertility').scale({zero: false}),
    v1.y().fieldQ('life_expect').scale({zero: false})
  )
  .render()

```

In some cases we might be unsatisfied with the default size range. To provide a customized span of sizes, set the range parameter of the scale attribute to an array indicating the smallest and largest sizes. Here we update the size encoding to range from 0 pixels (for zero values) to 1,000 pixels (for the maximum value in the scale domain):

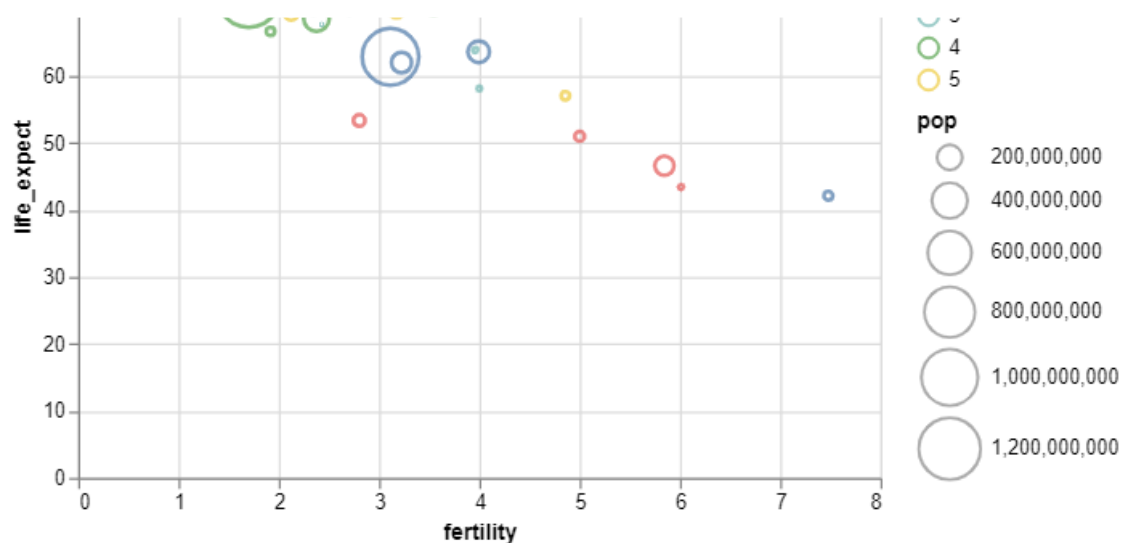




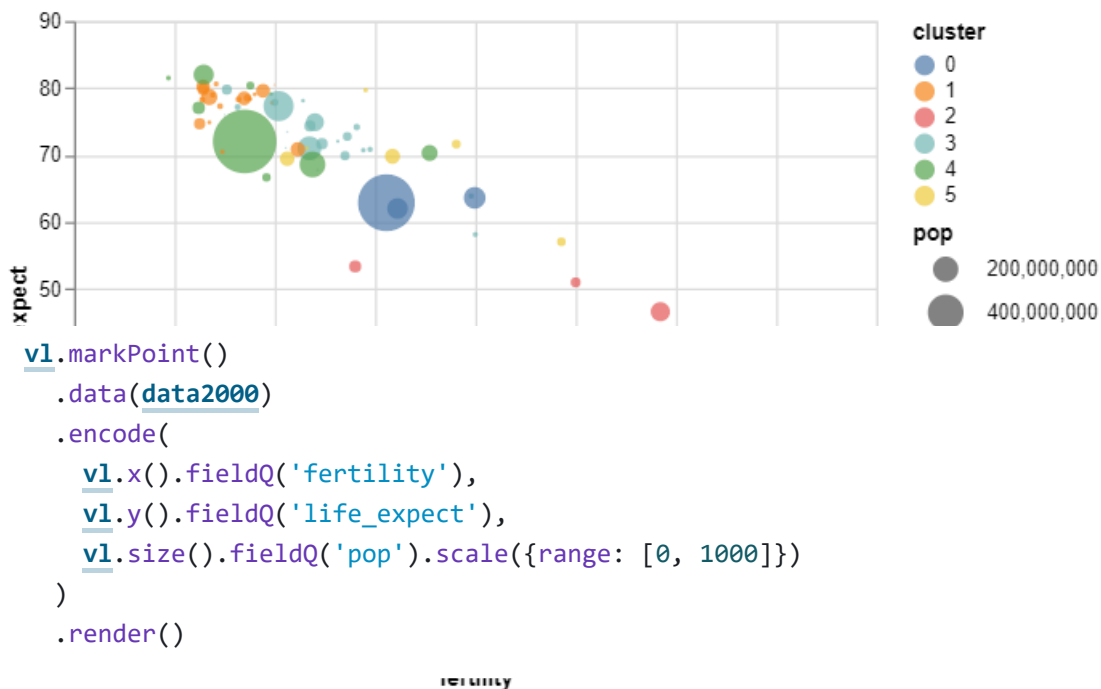
Color and Opacity

The color encoding channel sets a mark's color. The style of color encoding is highly dependent on the data type: nominal data will default to a multi-hued qualitative color scheme, whereas ordinal and quantitative data will use perceptually ordered color gradients.

```
v1.markPoint()
  .data(data2000)
  .encode(
    v1.x().fieldQ('fertility'),
    v1.y().fieldQ('life_expect'),
    v1.size().fieldQ('pop')
  )
  .render()
```

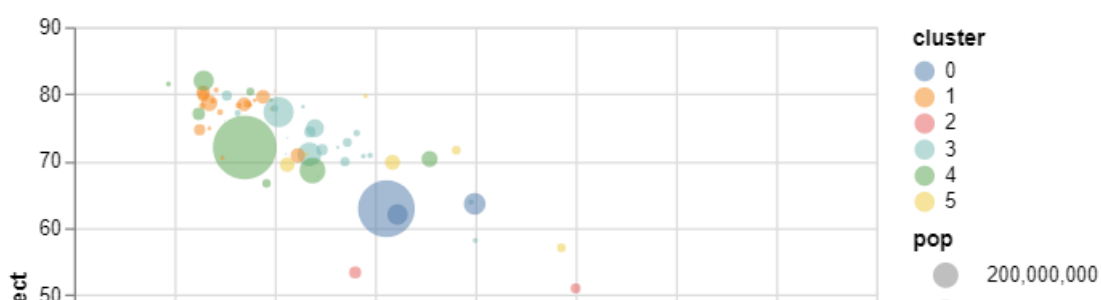


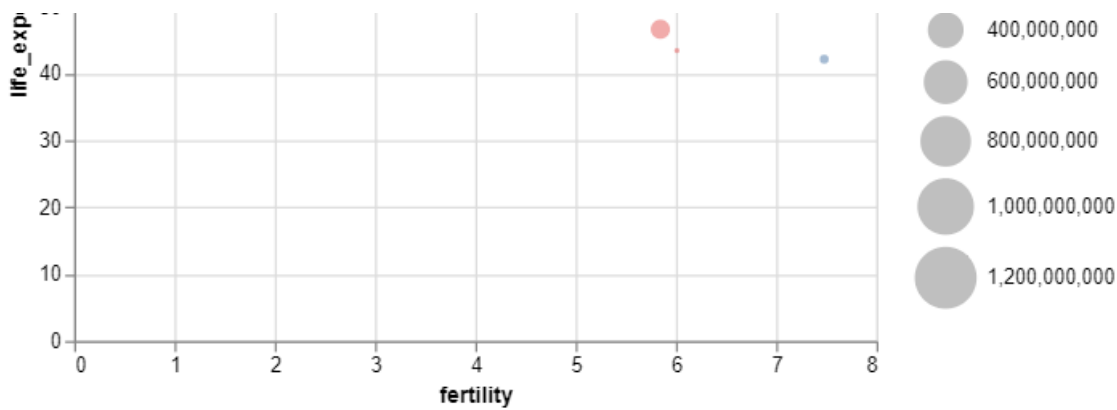
If you prefer filled shapes, pass `{filled: true}` to the `markPoint` method:



By default, Vega-Lite uses a bit of transparency to help combat over-plotting. We are free to further adjust the opacity, either by passing a default value to the `mark*` method, or using a dedicated encoding channel.

Here we demonstrate how to provide a constant value to an encoding channel instead of binding a data field:

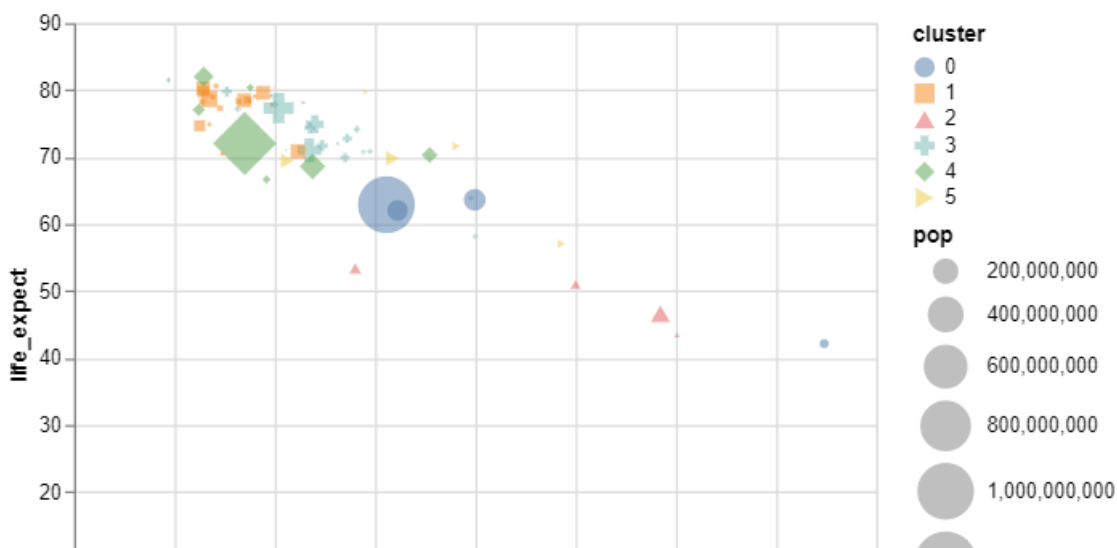




```
v1.markPoint()
  .data(data2000)
  .encode(
    v1.x().fieldQ('fertility'),
    v1.y().fieldQ('life_expect'),
    v1.size().fieldQ('pop').scale({range: [0, 1000]}),
    v1.color().fieldN('cluster')
  )
  .render()
```

rank-order and magnitude comparisons are not supported.

Let's encode the cluster field using shape as well as color. Using multiple channels for the same underlying data field is known as a *redundant encoding*. The resulting chart combines both color and shape information into a single symbol legend:



```
v1.markPoint({filled: true})
  .data(data2000)
  .encode(
    v1.x().fieldQ('fertility'),
```

```

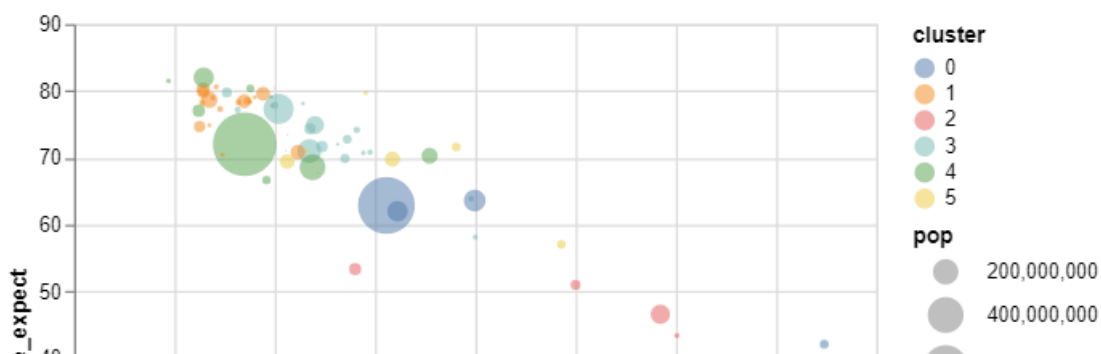
    vl.y().fieldQ('life_expect'),
    vl.size().fieldQ('pop').scale({range: [0, 1000]}),
    vl.color().fieldN('cluster')
  )
  .render()

```

Tooltips & Ordering

By this point, you might feel a bit frustrated: we've built up a chart, but we still don't know what countries the visualized points correspond to! Let's add interactive tooltips to enable exploration.

The tooltip encoding channel determines tooltip text to show when a user moves the mouse cursor over a mark. Let's add a tooltip encoding for the country field, then investigate which countries are being represented.



```

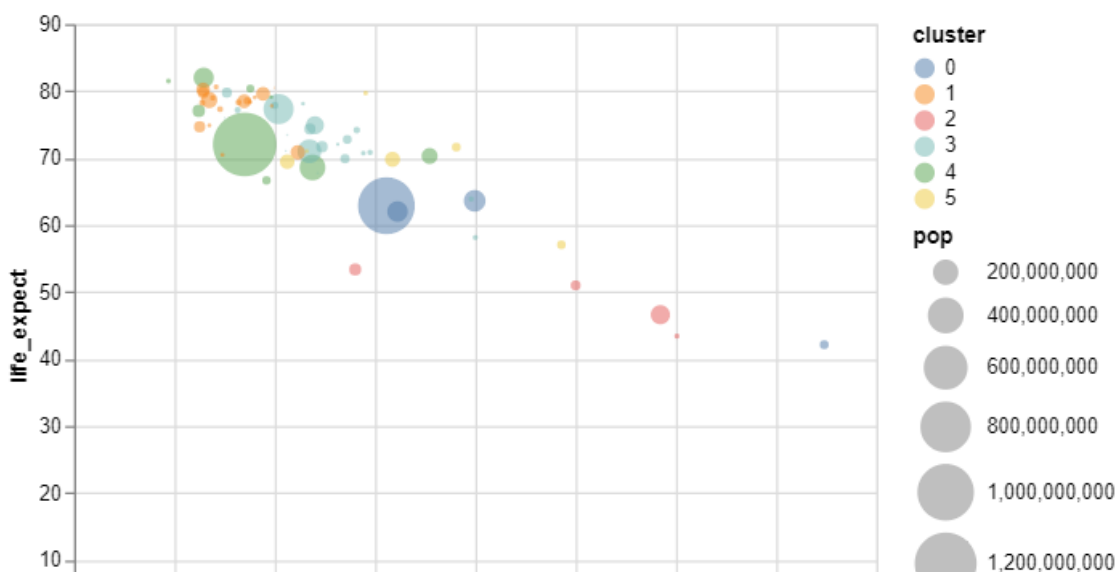
vl.markPoint({filled: true})
  .data(data2000)
  .encode(
    vl.x().fieldQ('fertility'),
    vl.y().fieldQ('life_expect'),
    vl.size().fieldQ('pop').scale({range: [0, 1000]}),
    vl.color().fieldN('cluster'),
    vl.opacity().value(0.5)
  )
  .render()

```

As you mouse around you may notice that you can not select some of the points. For example, the largest dark blue circle corresponds to India, which is drawn on top of a country with a smaller population, preventing the mouse from hovering over that country. To fix this problem, we can use the order encoding channel.

The order encoding channel determines the order of data points, affecting both the order in which they are drawn and, for line and area marks, the order in which they are connected to one another.

Let's order the values in descending rank order by the population (pop), ensuring that smaller circles are drawn later than larger circles:



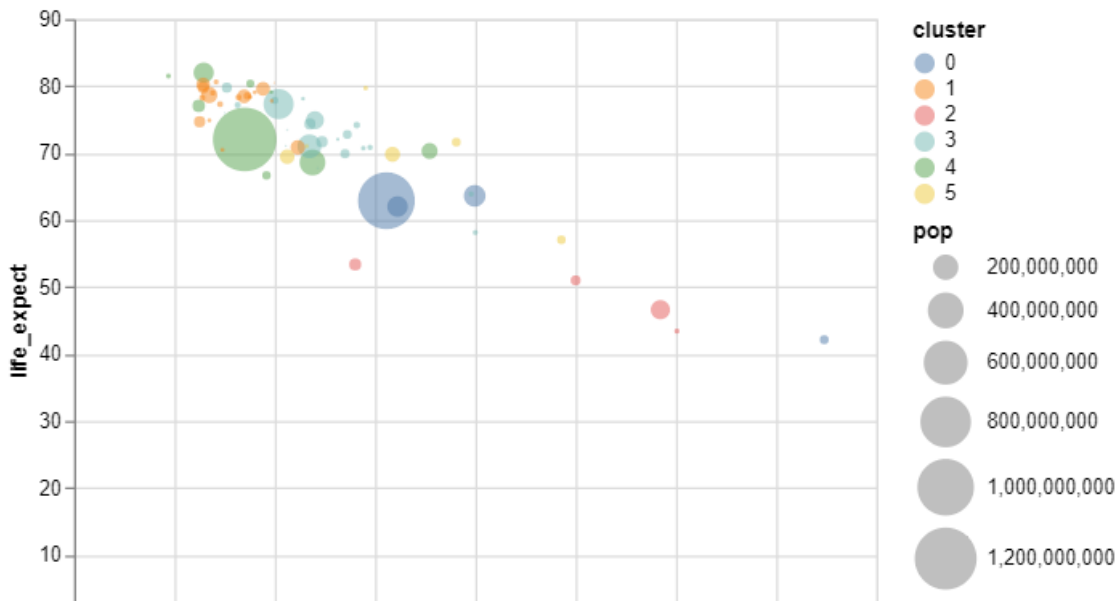
```
v1.markPoint({filled: true})
  .data(data2000)
  .encode(
    v1.x().fieldQ('fertility'),
    v1.y().fieldQ('life_expect'),
    v1.size().fieldQ('pop').scale({range: [0, 100]}),
    v1.color().fieldN('cluster'),
    v1.opacity().value(0.5),
    v1.shape().fieldN('cluster')
  )
  .render()
```

Now we can identify the smaller country being obscured by India: it's Bangladesh!

We can also now figure out what the cluster field represents. Mouse over the various colored points to formulate your own explanation.

At this point we've added tooltips that show only a single property of the underlying data record. To show multiple values, we can provide the tooltip channel an array of

encodings, one for each field we want to include:



```
v1.markPoint({filled: true})
  .data(data2000)
  .encode(
    v1.x().fieldQ('fertility'),
    v1.y().fieldQ('life_expect'),
    v1.size().fieldQ('pop').scale({range: [0, 1000]}),
    v1.color().fieldN('cluster'),
    v1.opacity().value(0.5),
    v1.tooltip().fieldN('country')
  )
  .render()
```

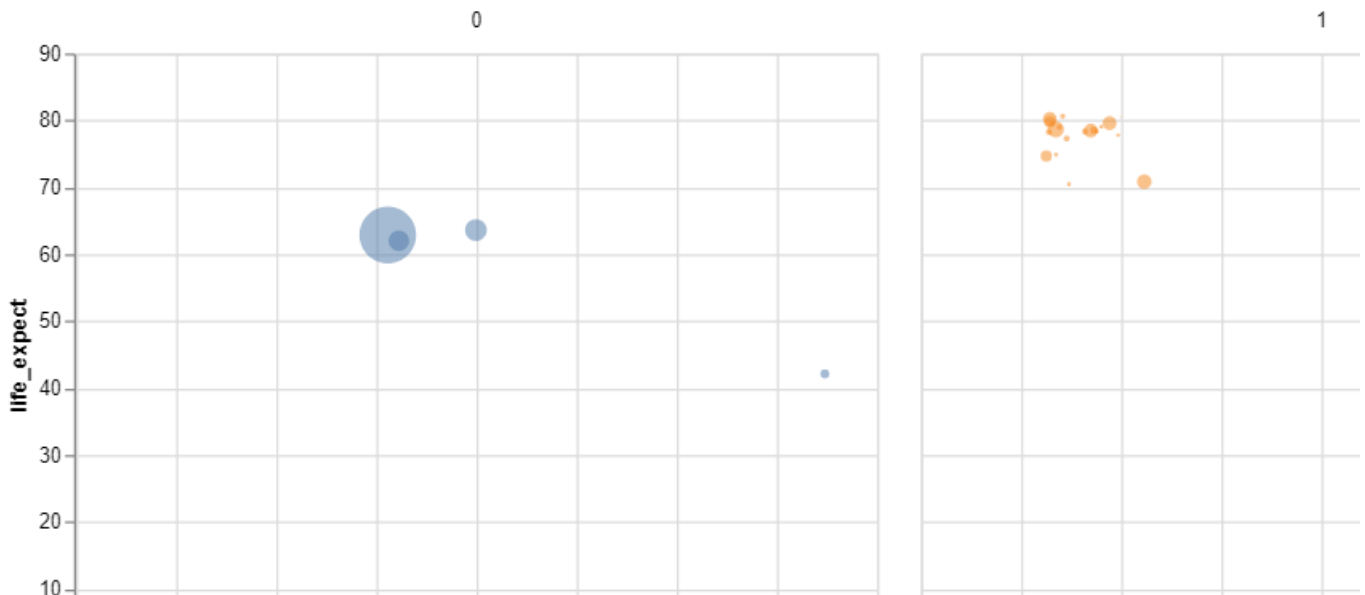
Now we can see multiple data fields upon mouse over! We used an array of string values to indicate the fields, which serves as a convenient alternative to writing `v1.tooltip().fieldN('name')` for all three field entries.

Column and Row Facets

Spatial position is one of the most powerful and flexible channels for visual encoding, but what can we do if we already have assigned fields to the x and y channels? One valuable technique is to create a *trellis plot*, consisting of sub-plots that show a subset of the data. A trellis plot is one example of the more general technique of presenting data using **small multiples** of views.

The column and row encoding channels generate either a horizontal (columns) or vertical (rows) set of sub-plots, in which the data is partitioned according to the provided data field.

Here is a trellis plot that divides the data into one column per cluster value:

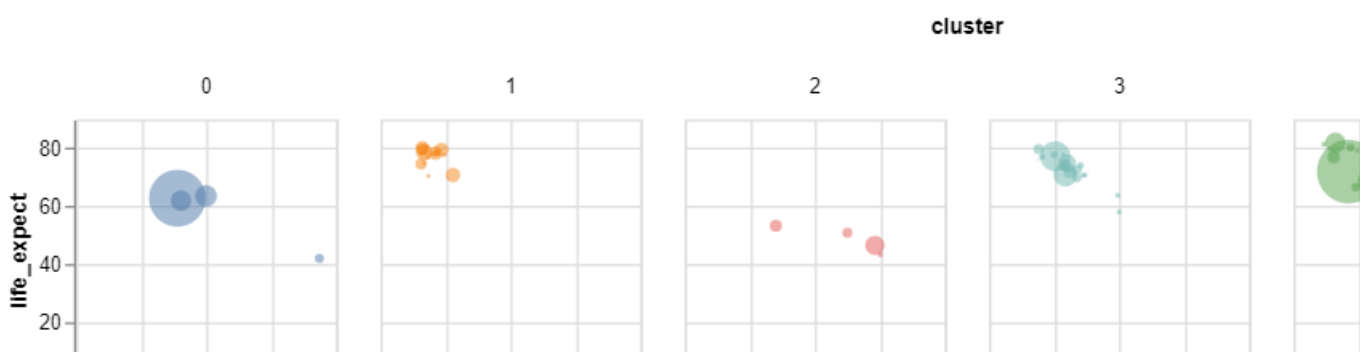


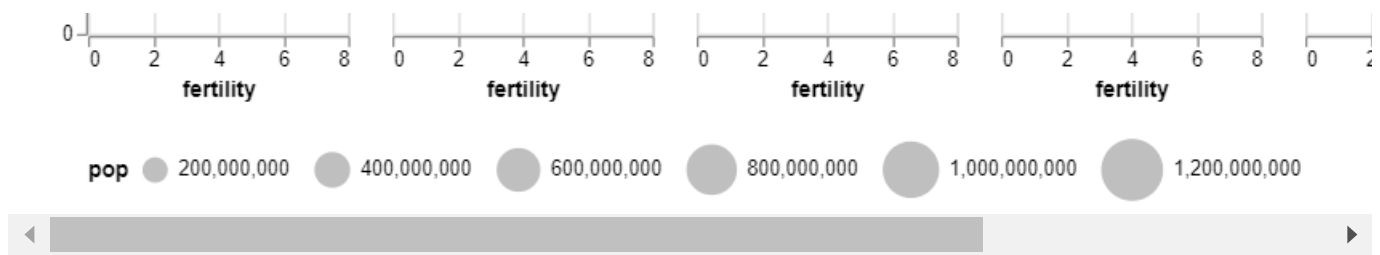
```

v1.markPoint({filled: true})
  .data(data2000)
  .encode(
    v1.x().fieldQ('fertility'),
    v1.y().fieldQ('life_expect'),
    v1.size().fieldQ('pop').scale({range: [0, 1000]}),
    v1.color().fieldN('cluster'),
    v1.opacity().value(0.5),
    v1.tooltip().fieldN('country'),
    v1.order().fieldQ('pop').sort('descending')
  )
  .render()

```

The plot above does not fit on screen, making it difficult to compare the sub-plots to each other! We can set the width and height properties to create a smaller set of multiples. Also, as the column headers already label the cluster values, let's remove our color legend by setting it to null. To make better use of space we can also orient our size legend to the 'bottom' of the chart.





```

v1.markPoint({filled: true})
  .data(data2000)
  .encode(
    v1.x().fieldQ('fertility'),
    v1.y().fieldQ('life_expect'),
    v1.size().fieldQ('pop').scale({range: [0, 1000]}),
    v1.color().fieldN('cluster'),
    v1.opacity().value(0.5),
    v1.order().fieldQ('pop').sort('descending'),
    v1.tooltip(['country', 'fertility', 'life_expect'])
  )
  .render()

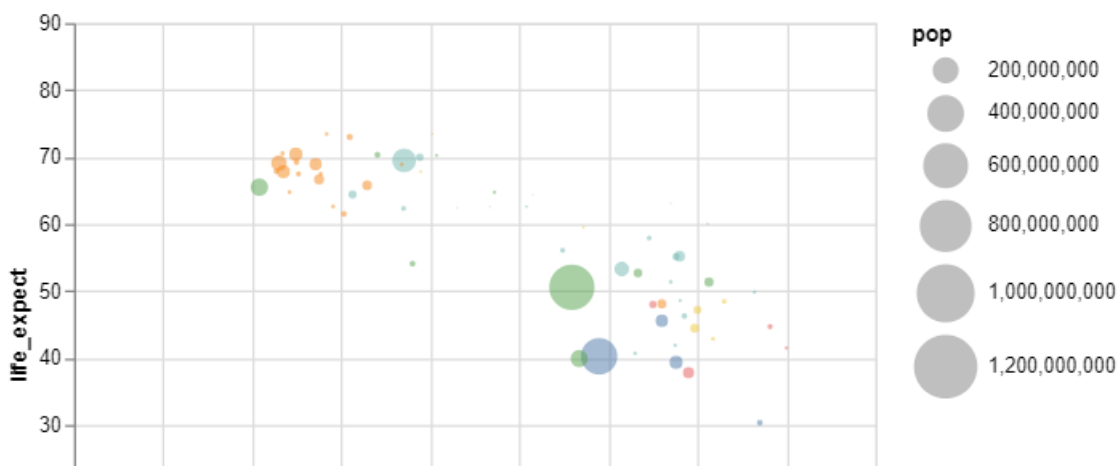
```

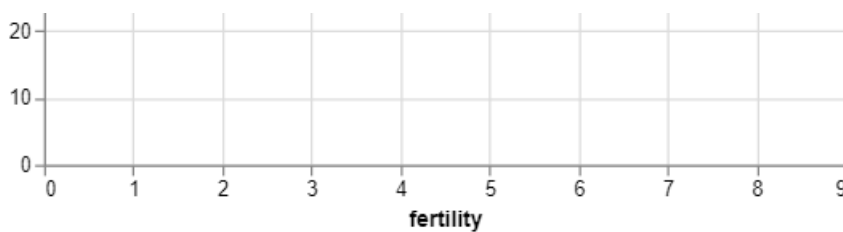
...the interactive, drag the slider back and forth to see how the data values change over time.

A Peek Ahead: Interactive Filtering

In later modules, we'll dive into interaction techniques for data exploration. Here is a sneak peak: binding a range slider to the year field to enable interactive scrubbing through each year of data. Don't worry if the code for the cell below is a bit confusing at this point, as we will cover interaction in detail later.

Drag the slider back and forth to see how the data values change over time!





select_year

Graphical Marks

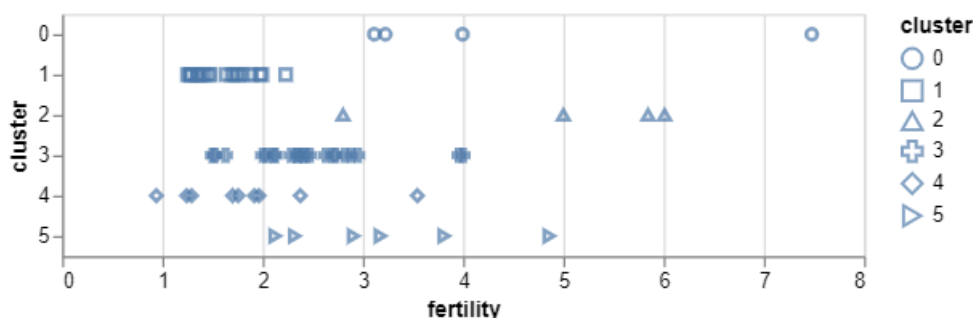
Our exploration of encoding channels above exclusively uses point marks to visualize the data. However, the point mark type is only one of the many geometric shapes that can be used to visually represent data. Vega-Lite includes a number of built-in mark types, including:

- `markArea()` - Filled areas defined by a top-line and a baseline.
- `markBar()` - Rectangular bars.
- `markCircle()` - Scatter plot points as filled circles.

```
v1.markPoint({filled: true})
  .data(data2000)
  .encode(
    v1.x().fieldQ('fertility'),
    v1.y().fieldQ('life_expect'),
    v1.size().fieldQ('pop').scale({range: [0, 1000]}),
    v1.color().fieldN('cluster'),
    v1.opacity().value(0.5),
    v1.tooltip().fieldN('country'),
    v1.order().fieldQ('pop').sort('descending'),
    v1.column().fieldN('cluster')
  )
  .render()
```

The point mark type conveys specific points, as in *scatter plots* and *dot plots*. In addition to x and y encoding channels (to specify 2D point positions), point marks can use color, size, and shape encodings to convey additional data fields.

Below is a dot plot of fertility, with the cluster field redundantly encoded using both the y and shape channels.



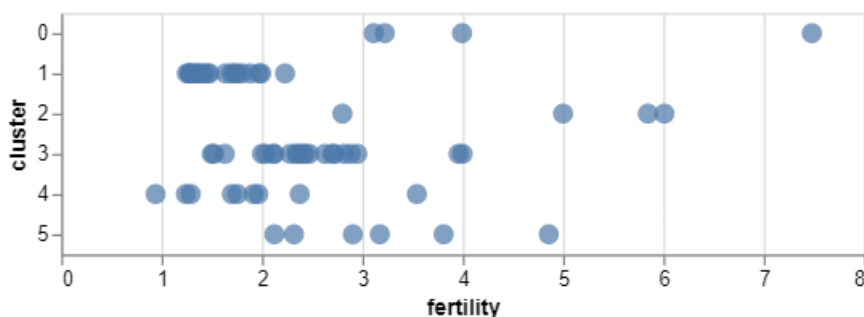
In addition to encoding channels, marks can be styled by providing values to the `mark*()` methods.

For example: point marks are drawn with stroked outlines by default, but can be specified to use filled shapes instead. Similarly, you can set a default size to set the total pixel area of the point mark.

```
v1.markPoint({filled: true})
  .data(data2000)
  .encode(
    v1.x().fieldQ('fertility'),
    v1.y().fieldQ('life_expect'),
    v1.size().fieldQ('pop').scale({range: [0, 1000]}),
    .legend({orient: 'bottom', titleOrient: 'left'}),
    v1.color().fieldN('cluster')
    .legend(null),
    v1.opacity().value(0.5),
    v1.tooltip().fieldN('country'),
    v1.order().fieldQ('pop').sort('descending'),
    v1.column().fieldN('cluster')
  )
  .width(130)
  .height(130)
  .render()
```

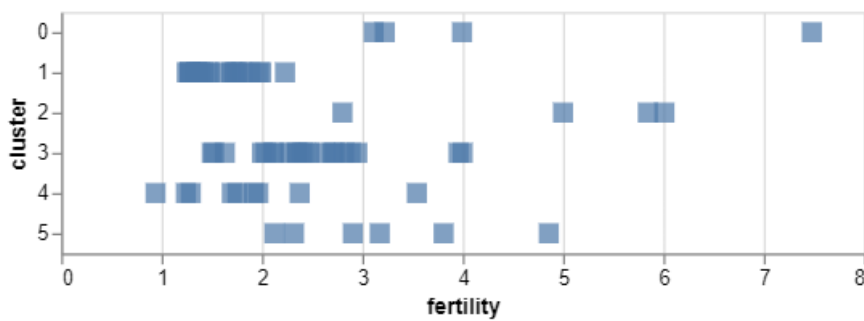
Circle Marks

The circle mark type is a shorthand for point marks drawn as filled circles.



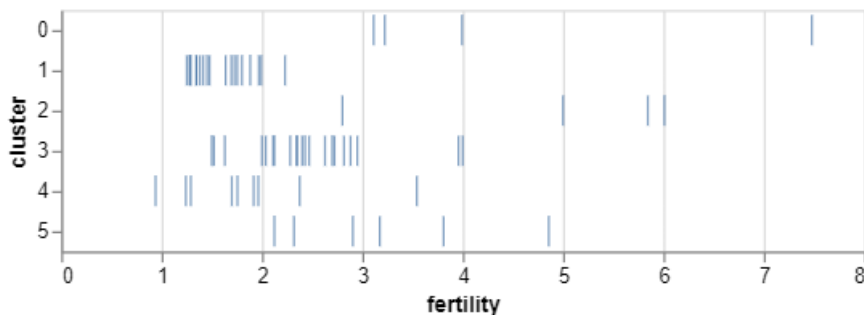
Square Marks

The square mark type is a shorthand for point marks drawn as filled squares.



Tick Marks

The tick mark type conveys a data point using a short line segment or "tick". These are particularly useful for comparing values along a single dimension with minimal overlap. A *dot plot* drawn with tick marks is sometimes referred to as a *strip plot*.

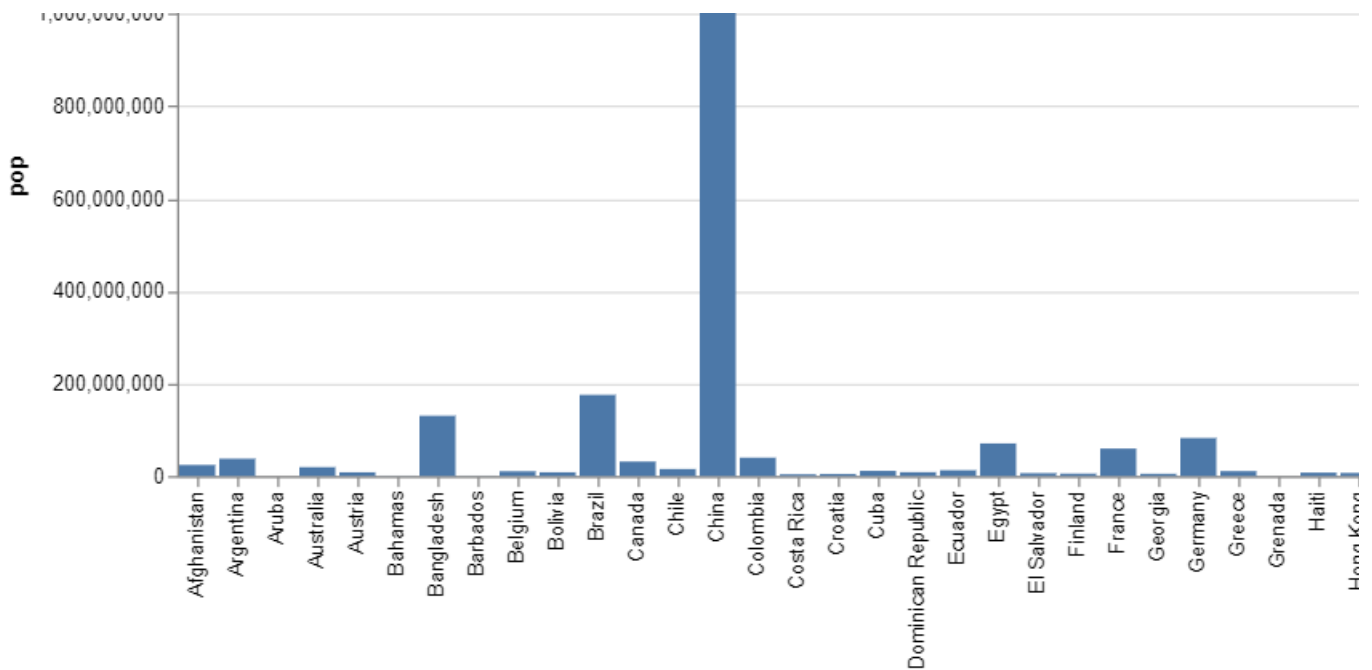


Bar Marks

The bar mark type draws a rectangle with a position, width, and height.

The plot below is a simple bar chart of the population (pop) of each country.





```

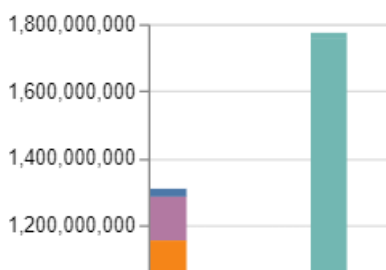
v1.markPoint()
  .data(data2000)
  .encode(
    v1.x().fieldQ('fertility'),
    v1.y().fieldN('cluster'),
    v1.shape().fieldN('cluster')
  )
  .render()

```

The bar width is set to a default size. *To change the width, try setting the step property of the markBar object's width attribute, like so:*

```
v1.markBar().width({step: 12}).
```

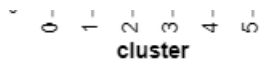
Bars can also be stacked. Let's change the x encoding to use the cluster field, and encode country using the color channel. We'll also disable the legend (which would be very long with colors for all countries!) and use tooltips for the country name.



```

v1.markPoint({filled: true, size: 100})
  .data(data2000)
  .encode(
    v1.x().fieldQ('fertility'),
    v1.y().fieldN('cluster'),
    v1.shape().fieldN('cluster')
  )
  .render()

```



```
v1.markCircle({size: 100})
  .data(data2000)
  .encode(
    v1.x().fieldQ('fertility'),
    v1.y().fieldN('cluster')
  )
  .render()
```

aggregation functions to determine the end points of the range. We will discuss aggregation in greater detail in the next notebook!

Alternatively, you can use `x` and `width` to provide a starting point plus offset, such that `x2 = x + width`.

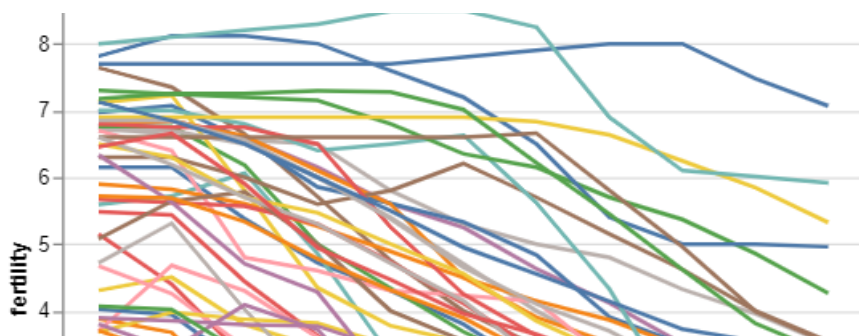


```
v1.markSquare({size: 100})
  .data(data2000)
  .encode(
    v1.x().fieldQ('fertility'),
    v1.y().fieldN('cluster')
  )
  .render()
```

Line Marks

The line mark type connects plotted points with line segments, for example so that a line's slope conveys information about the rate of change.

Let's plot a line chart of fertility per country over the years, using the full, unfiltered global development data frame. We'll again hide the legend and use tooltips instead.



```
v1.markTick()
  .data(data2000)
  .encode(
    v1.x().fieldQ('fertility'),
    v1.y().fieldN('cluster')
  )
  .render()
```

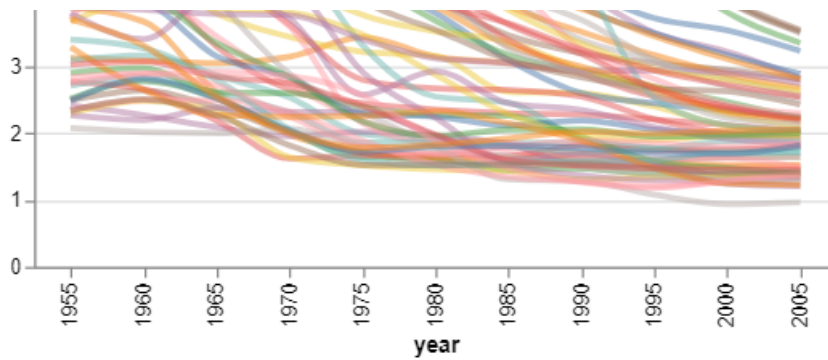
year

We can see interesting variations per country, but overall trends for lower numbers of children per family over time. Also note that we set a custom width of 400 pixels. *Try changing (or removing) the widths and see what happens!*

Let's change some of the default mark parameters to customize the plot. We can set the `strokeWidth` to determine the thickness of the lines and the `opacity` to add some transparency. By default, the line mark uses straight line segments to connect data points. In some cases we might want to smooth the lines. We can adjust the interpolation used to connect data points by setting the `interpolate` mark parameter. Let's use 'monotone' interpolation to provide smooth lines that are also guaranteed not to inadvertently generate "false" minimum or maximum values as a result of the interpolation.

```
9 |
v1.markBar()
  .data(data2000)
  .encode(
    v1.x().fieldN('country'),
    v1.y().fieldQ('pop')
  )
  .render()
```

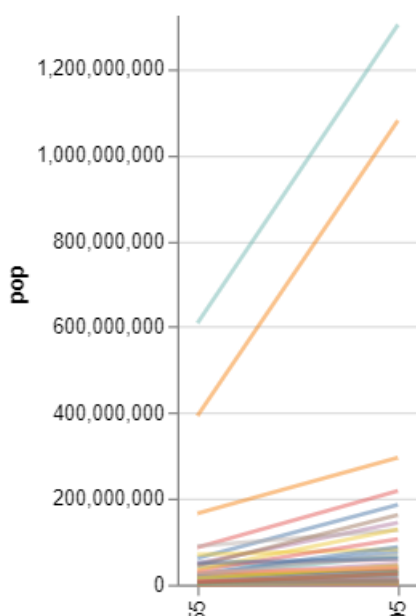




The line mark can also be used to create *slope graphs*, charts that highlight the change in value between two comparison points using line slopes.

Below let's create a slope graph comparing the populations of each country at minimum and maximum years in our full dataset: 1955 and 2005. We'll create a new data array filtered to those years, and use Vega-Lite to create the slope graph.

```
vl.markBar()
  .data(data2000)
  .encode(
    vl.x().fieldN('cluster'),
    vl.y().fieldQ('pop'),
    vl.color().fieldN('country').legend(null),
    vl.tooltip().fieldN('country')
  )
  .render()
```



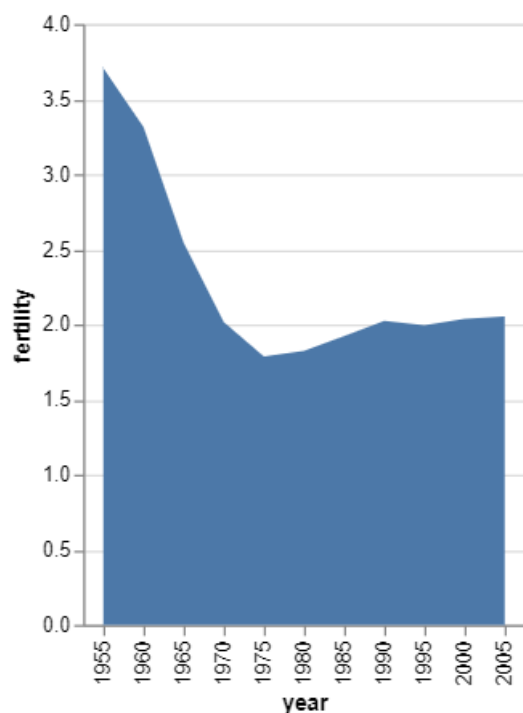
year

```
v1.markBar()
  .data(data2000)
  .encode(
    v1.x().min('life_expect'),
    v1.x2().max('life_expect'),
    v1.y().fieldN('cluster')
  )
  .render()
```

zero-valued baseline.

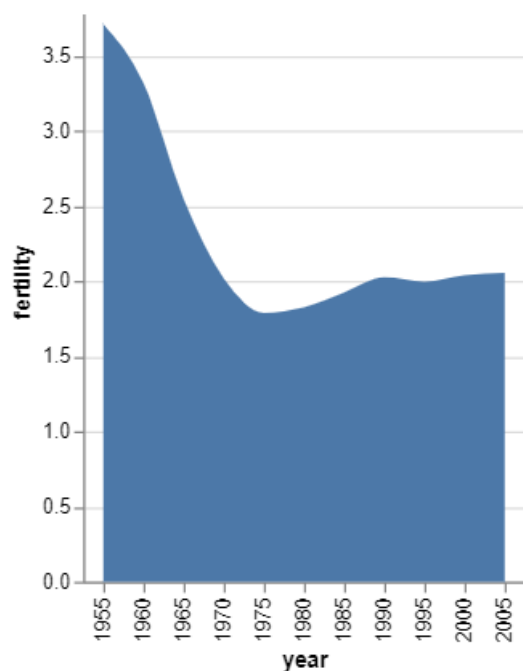
The chart below is an area chart of population over time for just the United States:

dataUS = ► Array(11) [Object, Object, Object, Object, Object, Object, Object, Object, Obj



```
v1.markLine()
  .data(data)
  .encode(
    v1.x().field0('year'),
    v1.y().fieldQ('fertility'),
    v1.color().fieldN('country').legend(null),
    v1.tooltip().fieldN('country')
  )
```

```
.width(400)
.render()
```

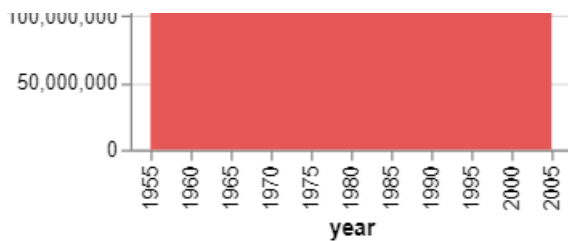


Similar to bar marks, area marks also support stacking. Here we create a new data frame with data for the three North American countries, then plot them using an area mark and a color encoding channel to stack by country.

```
dataNA = ► Array(33) [Object, Object, Object, Object, Object, Object, Object, Object, Obj
```

```
v1.markLine({strokeWidth: 3, opacity: 0.5, interpolate: 'monotone'})
.data(data)
.encode(
  v1.x().fieldQ('year'),
  v1.y().fieldQ('fertility'),
  v1.color().fieldN('country').legend(null),
  v1.tooltip().fieldN('country')
)
.width(400)
.render()
```

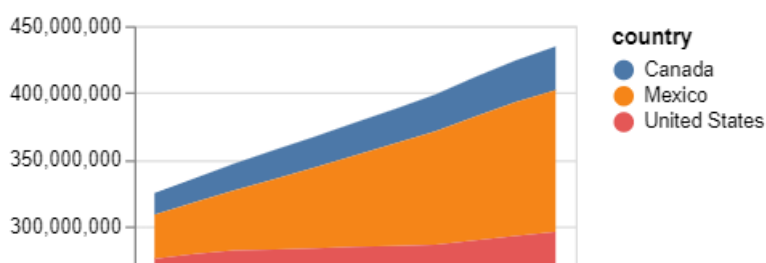




By default, stacking is performed relative to a zero baseline. However, other stack options are available:

- `center` - to stack relative to a baseline in the center of the chart, creating a *streamgraph* visualization, and
- `normalize` - to normalize the summed data at each stacking point to 100%, enabling percentage comparisons.

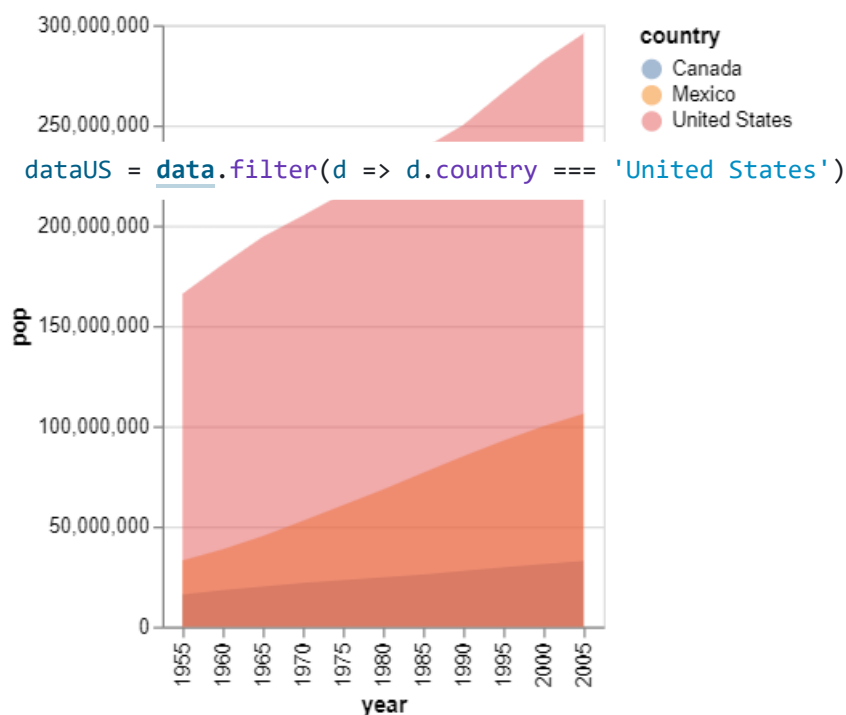
Below we adapt the chart by setting the y encoding stack property to center. *What happens if you instead set it `normalize`?*



```
v1.markLine({opacity: 0.5})
  .data(data.filter(d => d.year === 1955 || d.year === 2005))
  .encode(
    v1.x().fieldO('year').scale({padding: 0.1}),
    v1.y().fieldQ('pop'),
    v1.color().fieldN('country').legend(null),
    v1.tooltip().fieldN('country')
  )
  .width({step: 100})
  .render()
```

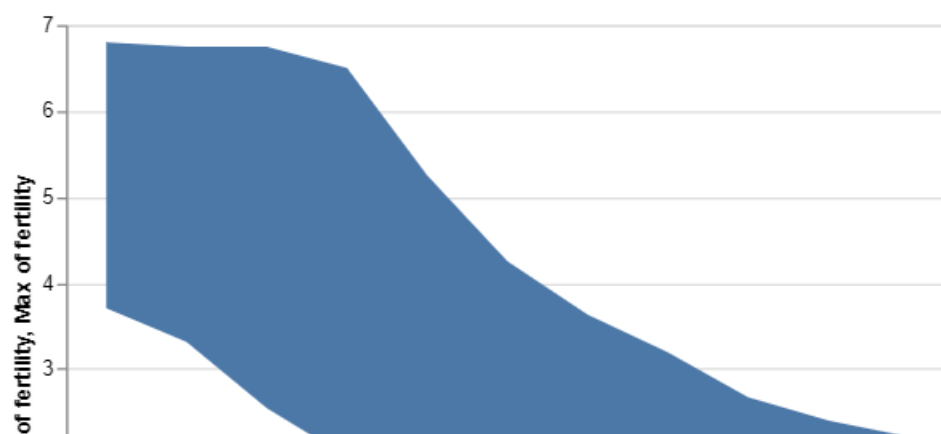
year

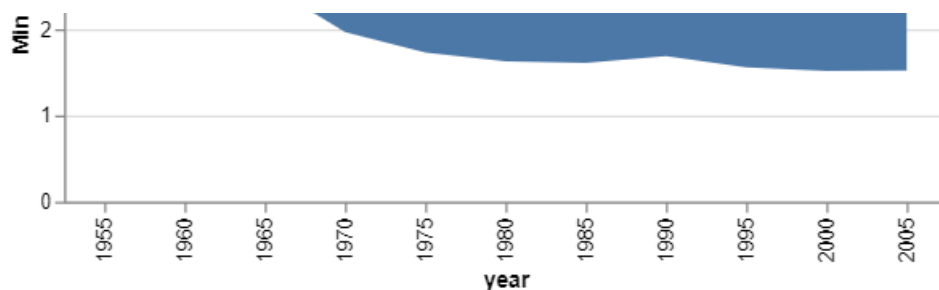
To disable stacking altogether, set the `stack` property to `null`. We can also add `opacity` as a default mark property to ensure we see the overlapping areas!



```
vl.markArea()
  .data(dataUS)
  .encode(
    vl.x().fieldQ('year'),
    vl.y().fieldQ('fertility')
  )
  .render()
```

The chart below visualizes the range of minimum and maximum fertility, per year, for North American countries:

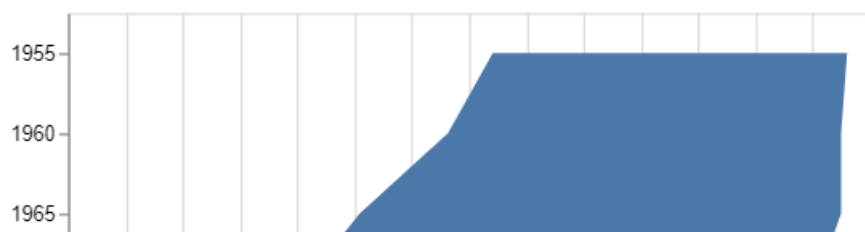




```

vl.markArea({interpolate: 'monotone'})
  .data(dataUS)
  .encode(
    vl.x().fieldQ('year'),
    vl.y().fieldQ('fertility')
  )
  .render()

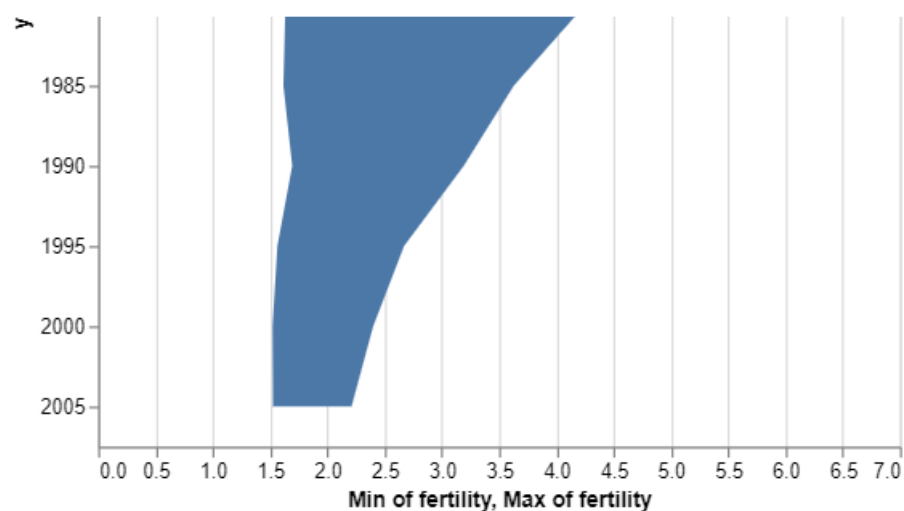
```



```

dataNA = data.filter(d => {
  return d.country === 'United States'
    || d.country === 'Canada'
    || d.country === 'Mexico';
})

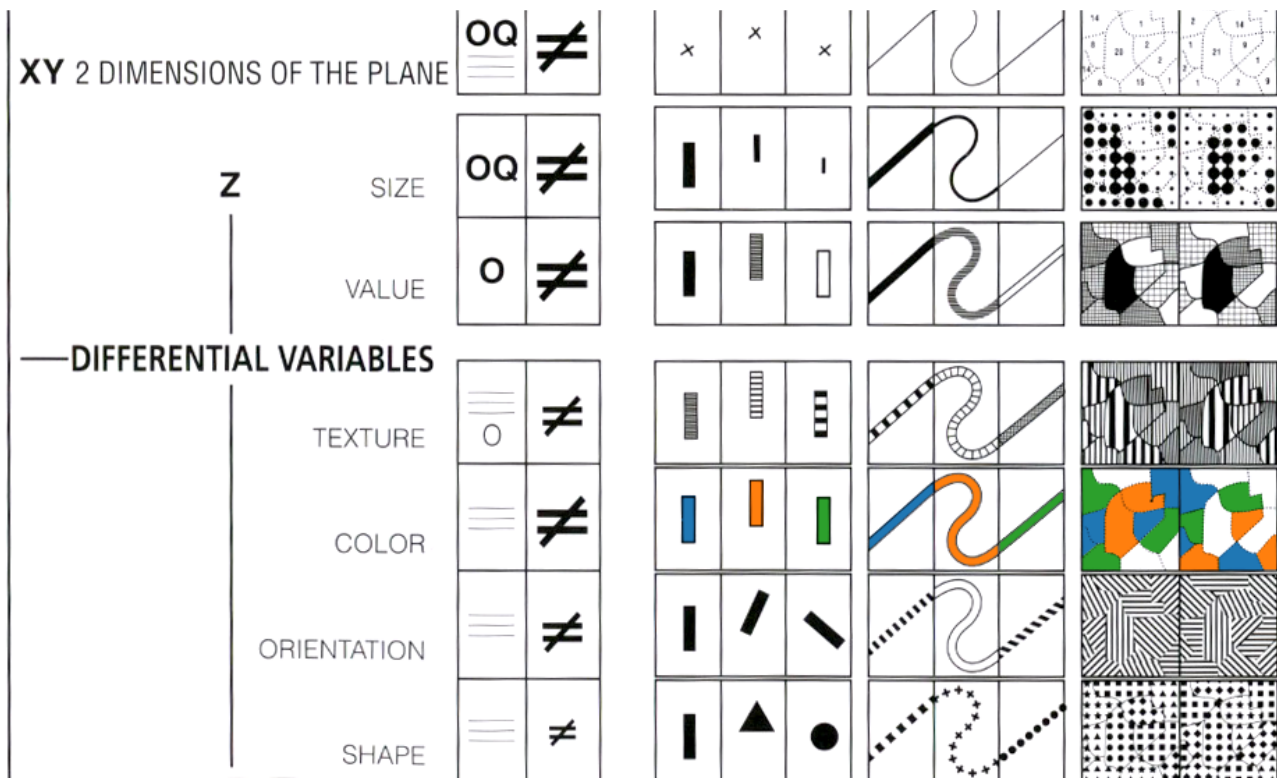
```



Summary

We've completed our tour of data types, encoding channels, and graphical marks! You should now be well-equipped to further explore the space of encodings, mark types, and mark parameters. For a comprehensive reference – including features we've skipped over here! – see the Vega-Lite [marks](#) and [encoding](#) documentation.

```
vl.markArea()
  .data(dataNA)
  .encode(
    vl.x().fieldO('year'),
    vl.y().fieldQ('pop'),
    vl.color().fieldN('country')
  )
  .render()
```



Bertin's taxonomy of visual encodings from *Sémiologie Graphique*, as adapted by Mike Bostock.

- The systematic study of marks, visual encodings, and backing data types was initiated by Jacques Bertin in his pioneering 1967 work *Sémiologie Graphique* (*The Semiology of Graphics*). The image above illustrates position, size, value (brightness),

texture, color (hue), orientation, and shape channels, alongside Bertin's recommendations for the data type comparisons they support.

- The framework of data types, marks, and channels also guides *automated* visualization design tools, starting with [Mackinlay's APT \(A Presentation Tool\)](#) in 1986 and continuing in more recent systems such as [Voyager](#) and [Draco](#).
- The identification of nominal, ordinal, interval, and ratio types dates at least as far back as S. S. Steven's 1947 article *On the theory of scales of measurement*.

```
v1.markArea()  
  .data(dataNA)  
  .encode(  
    v1.x().fieldO('year'),  
    v1.y().fieldQ('pop').stack('center'),  
    v1.color().fieldN('country')  
  )  
  .render()
```

```
v1.markArea({opacity: 0.5})  
  .data(dataNA)  
  .encode(  
    v1.x().fieldO('year'),  
    v1.y().fieldQ('pop').stack('center'),  
    v1.color().fieldN('country')
```

```
v1.x().fieldO('year'),  
v1.y().fieldQ('pop').stack(null),  
v1.color().fieldN('country')  
)  
.render()
```

```
v1.markArea()  
.data(dataNA)  
.encode(  
  v1.x().fieldO('year'),  
  v1.y().min('fertility'),  
  v1.y2().max('fertility')  
)  
.width({step: 40})  
.render();
```

```
v1.markArea()  
  .data(dataNA)  
  .encode(  
    v1.y().field0('year'),  
    v1.x().min('fertility'),  
    v1.x2().max('fertility')  
  )  
  .height({step: 40})  
  .render();
```

